



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 2 – Overloading and Inheritance
Topic – 2.6 Types of Inheritance

Prof. V.N.Nirgude
Assistant Professor

E-mail :

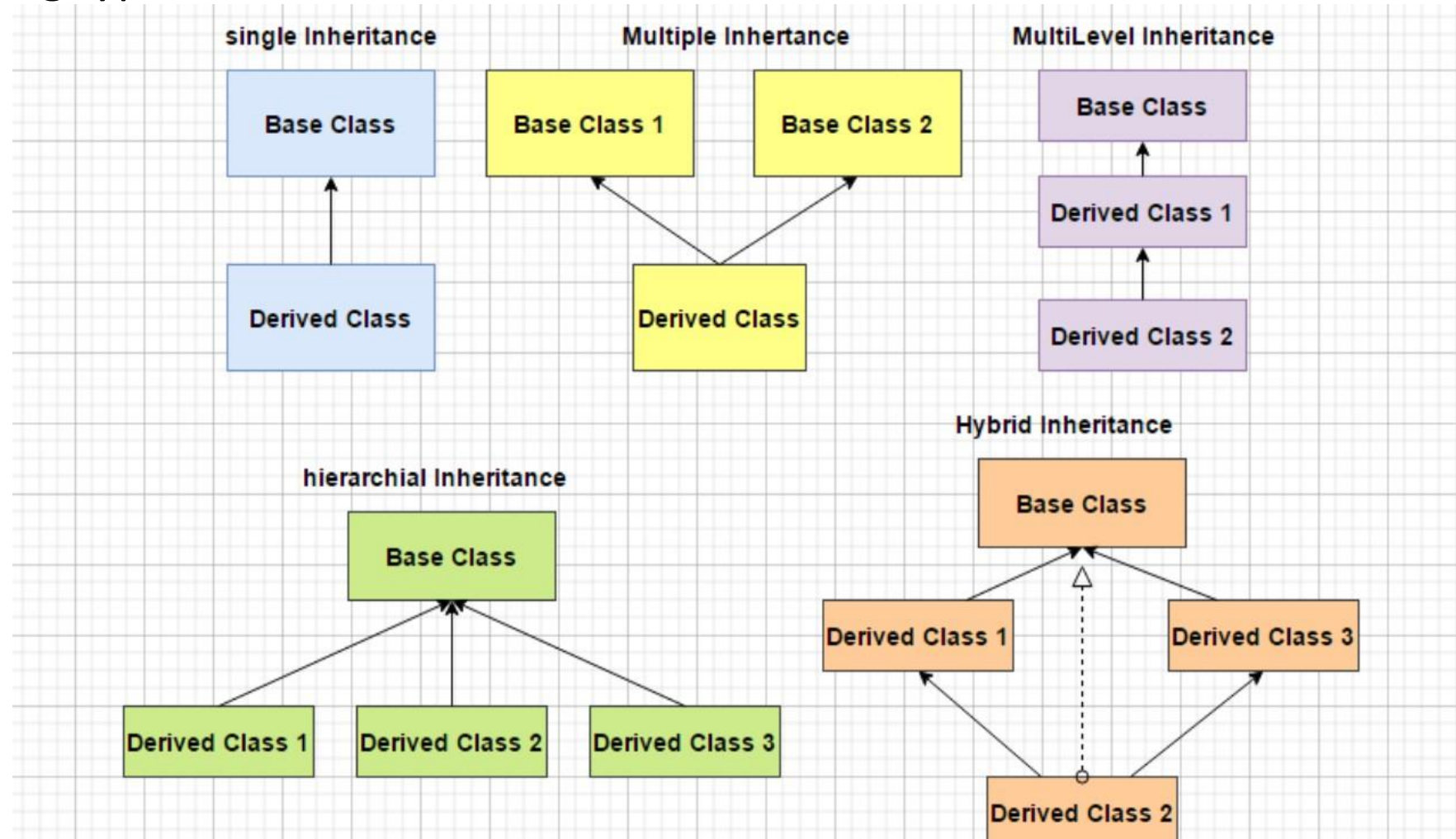
nirgudevikascomp@sanjivani.org.in

Contact No: 9975240215

Types of Inheritance

Inheritance are of the following types:

- Single Inheritance
- Multi level Inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance





Single Inheritance

- Single Inheritance is a process in which a sub class is derived from only one superclass.
- A class Car is derived from the class Vehicle

superclass(base class)

Vehicle



subclass(derived class)

Car

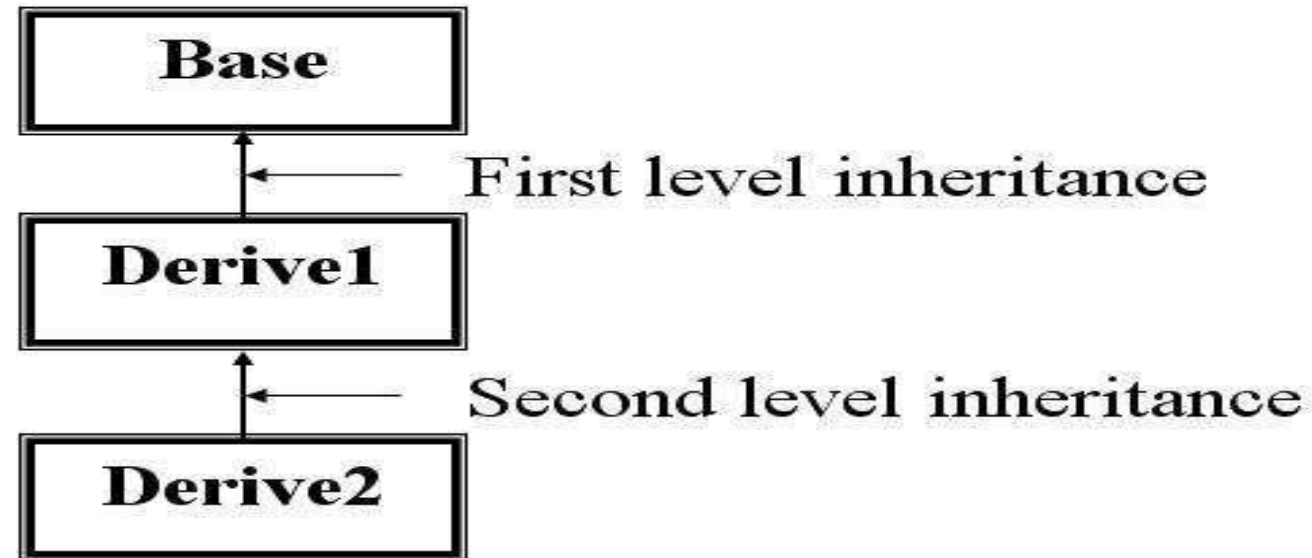
Defining the simple Inheritance

```
class vehicle
{
    ....
};
class car : visibility-mode vehicle
{
    .....
};
```



Multi level Inheritance

- C++ also provides the facility of multilevel inheritance, according to which the derived class can also be derived by an another class, which in turn can further be inherited by another and so on called as Multilevel or Varied Inheritance.



- In the above figure, class Base represents the base class. The class Derive1 that is called first level of inheritance, inherits the class Base.
- The derived class Derive1 is further inherited by the class Derive2, which is called second level of inheritance.



Example of Multi level Inheritance

```
#include <iostream>
using namespace std;
class base
{
public:
    int x;
    void getdata()
    {
        cout << "Enter value of x= ";
        cin >> x;
    }
};
class derive1 : public base
{
public:
    int y;
    void readdata()
```

```
{
    cout << "Enter value of y= ";
    cin >> y;
}
};
class derive2 : public derive1
{
private:
    int z;
public:
    void indata()
    {
        cout << "Enter value of z= ";
        cin >> z;
    }
    void product()
    {
```

```
        cout << "Product= " << x * y * z;
    }
};
int main()
{
    derive2 a;
    a.getdata();
    a.readdata();
    a.indata();
    a.product();
    return 0;
}
```

Output
Enter value of x= 2
Enter value of y= 3
Enter value of z= 3
Product= 18

Multiple Inheritance

- Deriving a class from more than one direct base classes is called *multiple inheritance*.

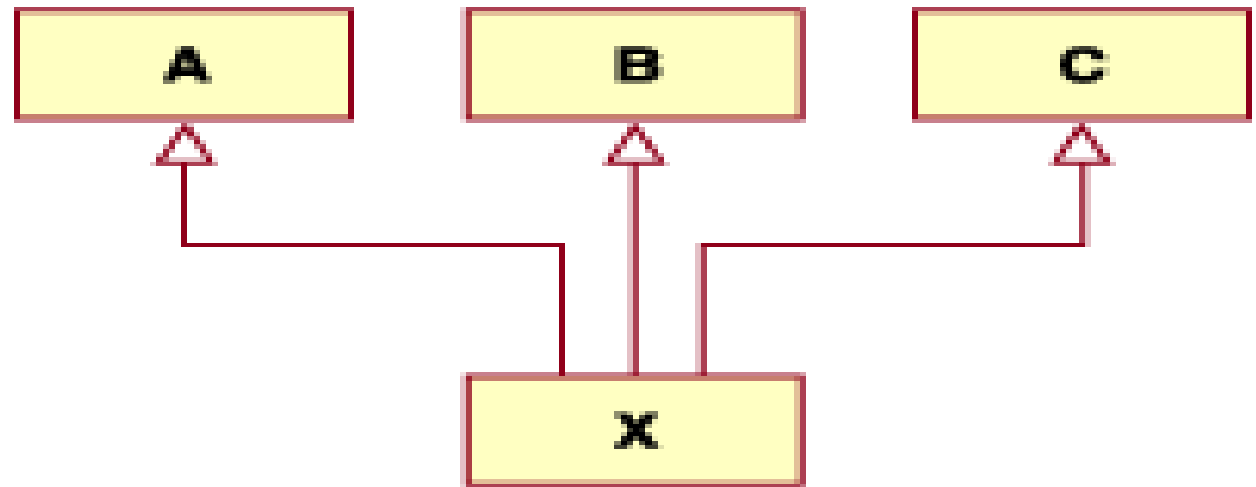
Defining the Multiple Inheritance

```
class A { /* ... */ };
```

```
class B { /* ... */ };
```

```
class C { /* ... */ };
```

```
class X :visibilty-mode A, visibilty-mode B, visibilty-mode C  
{ /* ... */ };
```





Example of Multiple Inheritance

class Circle // First base class

```
{
protected:
    float radius;
public:
    void Enter_r()
    {
        cout << "Enter the radius: ";
        cin >> radius;
    }
    void Display_CA()
    {
        cout << "Area = "<<(3.14 *radius*radius);
    }
};

class Rectangle // Second base class
{
protected:
    float length, breadth;
public:
    void Enter_lb()
    {
        cout << "Enter the length: ";
```

```
        cin >> length;
        cout << "Enter the breadth : ";
        cin >> breadth;
    }
    void Display_RA()
    {
        cout <<"Area =" <<(length * breadth);
    }
};

class Cylinder:public Circle,public Rectangle
{
public:
    void volume_cy()
    {
        cout << "Volume of cylinder is: "<<
        (3.14*radius*radius*length);
    }
};
```

int main()

```
{
    Cylinder c,r,cy;
    c.Enter_r();
    c.Display_CA();
    r.Enter_lb();
    r.Display_RA();
    cy.Enter_r();
    cy.Enter_lb();
    cy.volume_cy();
    return 0;
}

Output:
Enter the radius: 2
Area=12.56
Enter the length: 2
Enter the breadth : 3
Area= 6
Enter the radius: 5
Enter the length: 6
Enter the breadth : 7
Volume of cylinder is:471
```

Hierarchical Inheritance

- If a number of classes are derived from a single base class, it is called as hierarchical inheritance

- Defining the Hierarchical Inheritance :

Class A

{.....};

Class B: visibility-mode A

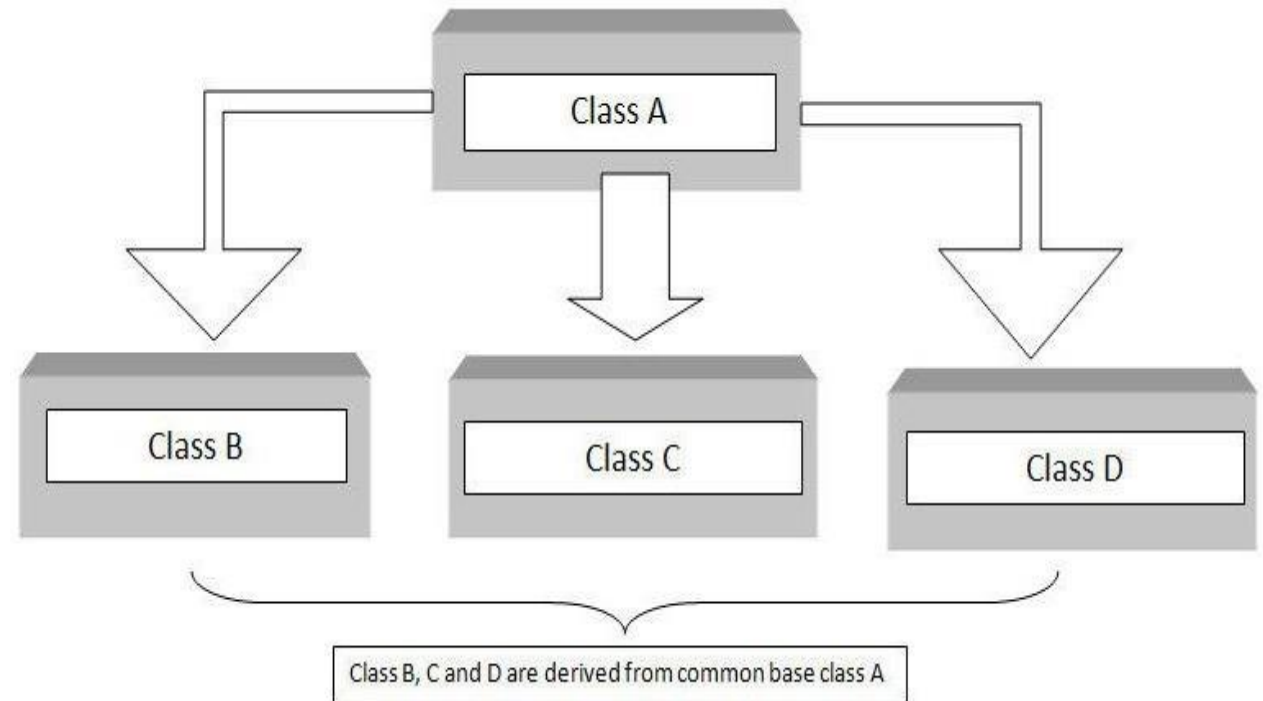
{.....};

Class C: visibility-mode A

{.....};

Class D: visibility-mode A

{.....};





Example using Hierarchical Inheritance

```
#include <iostream>
using namespace std;
class A
{
public:
    int x, y;
    void getdata()
    {
        cout << "Enter value of x & y:";
        cin >> x >> y;
    }
};
class B : public A
{
public:
    void product()
    {
        cout << "Product= " << x * y;
    }
};
```

```
class C : public A
{
public:
    void sum()
    {
        cout << "Sum= " << x + y;
    }
};
int main()
{
    B obj1;
    C obj2;
    obj1.getdata();
    obj1.product();
    obj2.getdata();
    obj2.sum();
    return 0;
}
```

Output

```
Enter value of x & y: 2 3
Product= 6
Enter value of x & y: 2 3
Sum= 5
```



Hybrid Inheritance

- In this type, more than one type of inheritance are used to derive a new sub-class.
- Multiple and Multilevel type of inheritances are used to derive a class PG-Student.

```
class Person
```

```
{ .....};
```

```
class Student : public Person
```

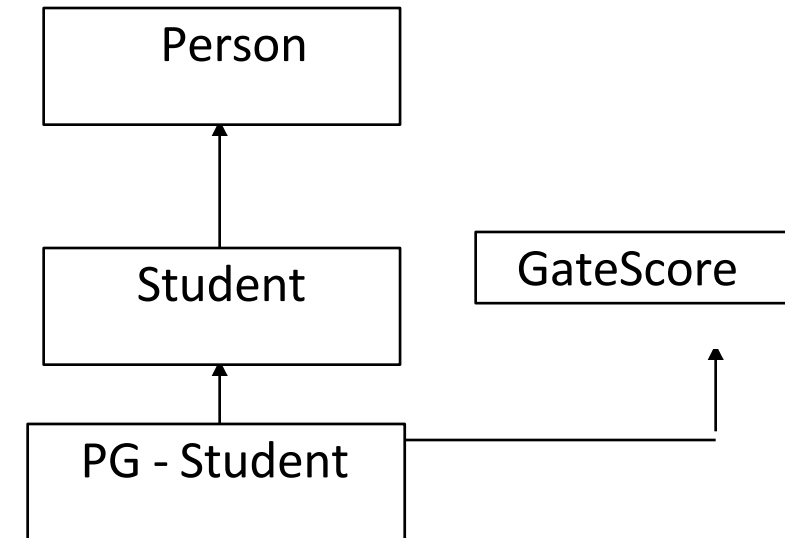
```
{ .....};
```

```
class GateScore
```

```
{.....};
```

```
class PG-Student : public Student, public GateScore
```

```
{.....};
```



Thank you..